

Term Project — Interim Report

Graph-Based Bug Prediction Using Structural, Co-change, and Semantic Signals

1. Project Overview

The goal of this project is to predict which modules in a software project are likely to contain bugs by representing source code repositories as graphs and applying graph theoretic analysis. Three public Python repositories, requests, flask, and scikit-learn, are used as data sources. Each repository is modeled as a combination of three distinct graph layers, and a composite risk score is computed for each module based on its structural position across all three layers.

2. Research Question

Can the combination of a module's structural position in the dependency graph, its co-change history, and the semantic content of its associated commit messages produce more accurate bug predictions than any single signal used in isolation?

3. Graph Model

The project builds three separate graph representations from each repository:

- **Dependency Graph (directed):** Nodes are source files. A directed edge $A \rightarrow B$ exists if module A imports module B. This graph captures the structural architecture of the codebase.
- **Co-change Graph (undirected, weighted):** An edge between A and B is added every time both files appear in the same commit. Edge weight reflects the frequency of co-change. This graph captures behavioural coupling.
- **Semantic Similarity Graph (undirected, weighted):** For each module, all associated commit messages are concatenated and represented as a TF-IDF vector. An edge between two modules is added if their cosine similarity exceeds a defined threshold. This graph captures thematic relatedness through natural language.

4. Graph Theory Methods Applied

- **Betweenness Centrality:** Applied to the dependency graph to identify modules that act as bridges between different parts of the codebase.
- **PageRank:** Applied to the dependency graph to find modules that are depended upon by many other important modules.
- **Community Detection (Louvain):** Applied to the co-change graph to identify clusters of files that are frequently changed together, suggesting functional coupling.
- **Weighted Degree Centrality:** Applied to both the co-change and semantic graphs to measure how extensively each module is connected to others.

5. NLP Integration

Commit messages are used as a lightweight natural language signal. For each source file, all commit messages from commits that touched that file are collected and concatenated into a single text document. These documents are then vectorized using TF-IDF (Term Frequency–Inverse Document Frequency) with bigram support. Pairwise cosine similarity between all module documents is computed to construct the semantic similarity graph. Files whose associated commit messages share

recurring terminology such as error types, subsystem names, or fix patterns will appear as neighbours in this graph, revealing thematic relationships not visible in the import structure.

6. Data Collection

Data is collected entirely from public GitHub repositories via the GitHub REST API. The following three repositories were selected based on commit volume, project maturity, and domain diversity:

- `psf/requests` — HTTP library, relatively small and well-structured
- `pallets/flask` — Web framework, moderate size and complexity
- `scikit-learn/scikit-learn` — Machine learning library, large codebase with long commit history

For each repository, the following data was collected for the period 2022–present:

- Full commit metadata: SHA, author, date, and commit message
- Per-commit file change lists: filename, lines added, lines deleted, change status
- Bug-fix labels: commits containing keywords such as `fix`, `bug`, `error`, `crash`, `patch`, `resolve` were tagged as bug-fixing commits

7. Current Progress

- **Completed:** GitHub API integration with rate limit handling and checkpoint-based resumption
- **Completed:** Commit and file-level data collected for all three repositories
- **Completed:** Data cleaning pipeline: `test/docs` files filtered out, bug-fix commits labelled, per-file statistics computed
- **In progress:** Construction of the three graph layers (dependency, co-change, semantic)
- **Pending:** Centrality computation, risk score aggregation, and evaluation against ground truth labels

8. Known Limitations

- Bug-fix identification relies on commit message keywords, which are inconsistent across developers. Some bug fixes may be committed without explicit labelling and will not be captured.
- The approach is suited to repositories with a long and active commit history. Sparse repositories may not contain enough signal for meaningful predictions.
- The semantic graph uses TF-IDF rather than contextual embeddings. This may miss semantic relationships expressed through varied vocabulary.
- The system can only reason about modules with existing commit history. Newly written files with no prior commits cannot be evaluated.